Pack 14 — API Audit Logging, Compliance Exports & Support Ticket Persistence (real code)

RemoteDesk · Pack 14 — API Audit Logging, Compliance Exports & Support Ticket Persistence (real code)

> Codex / Claude Code handoff. Copy each file to the path in its heading. Build order: shared → api → web → desktop → tests. Every file is tagged **SAFE_DIRECT_COPY** (new file, drop in) or **REVIEW_REQUIRED** (patch to an existing file — merge by hand).
> Scope: append-only API audit logging with mandatory redaction, secret-free compliance CSV exports, and persisted support tickets (public vs internal comments, status + priority). **No live email sending. No fake admin/RBAC. Native input execution stays disabled. Audit logs and exports redact Authorization headers, cookies, passwords, tokens, API keys, and device passwords.**

Zip-ready file tree

```
remotedesk/
  packages/shared/src/
    audit/
      types.ts                          # SAFE_DIRECT_COPY
      redaction.ts                      # SAFE_DIRECT_COPY
      exportCsv.ts                      # SAFE_DIRECT_COPY
      schema.ts                         # SAFE_DIRECT_COPY
      index.ts                          # SAFE_DIRECT_COPY
    support/
      types.ts                          # SAFE_DIRECT_COPY
      status.ts                         # SAFE_DIRECT_COPY
      priority.ts                       # SAFE_DIRECT_COPY
      schema.ts                         # SAFE_DIRECT_COPY
      index.ts                          # SAFE_DIRECT_COPY
    index.ts                            # REVIEW_REQUIRED (patch)
    errors/errorCodes.ts                # REVIEW_REQUIRED (patch)
  apps/api/
    prisma/schema.prisma                # REVIEW_REQUIRED (patch)
    prisma/migrations/0014_audit_support/migration.sql   # SAFE_DIRECT_COPY
    src/lib/auditLogger.ts              # SAFE_DIRECT_COPY
    src/lib/complianceExports.ts        # SAFE_DIRECT_COPY
    src/lib/supportTickets.ts           # SAFE_DIRECT_COPY
    src/repositories/auditRepository.ts          # SAFE_DIRECT_COPY
    src/repositories/supportRepository.ts        # SAFE_DIRECT_COPY
    src/routes/audit.routes.ts          # SAFE_DIRECT_COPY
    src/routes/support.routes.ts        # SAFE_DIRECT_COPY
    src/server.ts                       # REVIEW_REQUIRED (patch)
  apps/web/app/dashboard/
    audit/page.tsx                      # SAFE_DIRECT_COPY
    audit/auditApi.ts                   # SAFE_DIRECT_COPY
    support/page.tsx                    # SAFE_DIRECT_COPY
```

```
    support/supportApi.ts              # SAFE_DIRECT_COPY
    support/[ticketId]/page.tsx        # SAFE_DIRECT_COPY
    support/support.module.css         # SAFE_DIRECT_COPY
    page.tsx                           # REVIEW_REQUIRED (patch — add links)
  apps/desktop/src/renderer/src/
    services/auditClient.ts            # SAFE_DIRECT_COPY
    services/supportClient.ts          # SAFE_DIRECT_COPY
    session/criticalEvents.ts          # REVIEW_REQUIRED (patch)
  tests/audit/
    redaction.test.ts                  # SAFE_DIRECT_COPY
    exportCsv.test.ts                  # SAFE_DIRECT_COPY
    audit-contract.test.ts             # SAFE_DIRECT_COPY
  tests/support/
    status.test.ts                     # SAFE_DIRECT_COPY
    priority.test.ts                   # SAFE_DIRECT_COPY
    support-ticket-contract.test.ts    # SAFE_DIRECT_COPY
  docs/
    audit-compliance-support.md        # SAFE_DIRECT_COPY
  generated-remotedesk-pack14-merge-summary.md   # doNotMerge (handoff meta)
  generated-remotedesk-pack14-code-review.md     # doNotMerge (handoff meta)
  generated-remotedesk-pack14-manifest.json      # doNotMerge (handoff meta)
```

Part A — Shared package: audit

`packages/shared/src/audit/types.ts` — SAFE_DIRECT_COPY

```ts
/** Critical, audit-worthy action categories. */
export type AuditAction =
  | 'auth.login'
  | 'auth.logout'
  | 'session.start'
  | 'session.end'
  | 'file.transfer.attempted'
  | 'clipboard.sync.toggled'
  | 'input.permission.changed'
  | 'audit.export.created'
  | 'support.ticket.created'
  | 'support.ticket.status.changed';

/** Actor that performed the action. */
export interface AuditActor {
  userId: string | null;
  /** Optional device the action originated from. */
  deviceId?: string | null;
}

/** An append-only audit log entry. Metadata is ALWAYS redacted before storage. */
export interface AuditLogDto {
  id: string;
  orgId: string;
  action: AuditAction;
  actor: AuditActor;
  /** Redacted, content-light key/value context. */
  metadata: Record<string, string>;
```

```
  /** ISO timestamp. */
  createdAt: string;
}

/** Input for creating an audit log (server redacts metadata). */
export interface AuditLogInput {
  action: AuditAction;
  userId?: string | null;
  deviceId?: string | null;
  metadata?: Record<string, string>;
}

/** A compliance export job over a date range. */
export interface ComplianceExportDto {
  id: string;
  orgId: string;
  fromDate: string;
  toDate: string;
  rowCount: number;
  /** Status of the export generation. */
  status: 'ready' | 'empty';
  createdAt: string;
}
```

`packages/shared/src/audit/redaction.ts` — SAFE_DIRECT_COPY

```
/**
 * Pure audit redaction shared by server (authoritative, runs before persistence),
 * desktop client (defense in depth), and tests. Masks secret-bearing keys —
 * including DEVICE PASSWORDS — and never returns the original value.
 */
export const REDACTION_MASK = '[REDACTED]';

export const SENSITIVE_KEY_PATTERNS: readonly RegExp[] = [
  /authorization/i,
  /cookie/i,
  /set-cookie/i,
  /token/i,
  /password/i,        // covers "devicePassword", "password", "passwd"
  /passwd/i,
  /secret/i,
  /api[-_]?key/i,
  /access[-_]?key/i,
  /refresh[-_]?token/i,
  /bearer/i,
  /private[-_]?key/i,
  /device[-_]?password/i,
];

export function isSensitiveKey(key: string): boolean {
  return SENSITIVE_KEY_PATTERNS.some((re) => re.test(key));
}

/** Redact a flat string map. Returns a scrubbed copy; never mutates the input. */
export function redactMetadata(
  metadata: Record<string, string>,
): Record<string, string> {
```

```
  const out: Record<string, string> = {};
  for (const [key, value] of Object.entries(metadata)) {
    out[key] = isSensitiveKey(key) ? REDACTION_MASK : value;
  }
  return out;
}

/** Defense-in-depth backstop: throw if a sensitive key survived unmasked. */
export function assertRedacted(metadata: Record<string, string>): void {
  for (const [key, value] of Object.entries(metadata)) {
    if (isSensitiveKey(key) && value !== REDACTION_MASK) {
      throw new Error(`Unredacted sensitive audit key: "${key}"`);
    }
  }
}
```

`packages/shared/src/audit/exportCsv.ts` — SAFE_DIRECT_COPY

```
import type { AuditLogDto } from './types';
import { redactMetadata } from './redaction';

/**
 * Pure CSV serialization for compliance exports. Re-redacts metadata as a final
 * safety net so an export can NEVER contain secrets even if a raw row slipped
 * through. Values are RFC-4180 quoted.
 */
const COLUMNS = ['id', 'createdAt', 'action', 'userId', 'deviceId', 'metadata'] as const;

export function auditLogsToCsv(logs: AuditLogDto[]): string {
  const header = COLUMNS.join(',');
  const rows = logs.map((log) => {
    const safeMeta = redactMetadata(log.metadata);
    const cells = [
      log.id,
      log.createdAt,
      log.action,
      log.actor.userId ?? '',
      log.actor.deviceId ?? '',
      JSON.stringify(safeMeta),
    ];
    return cells.map(csvQuote).join(',');
  });
  return [header, ...rows].join('¥r
');
}

/** RFC-4180 quoting: wrap in quotes, double embedded quotes. */
export function csvQuote(value: string): string {
  const v = String(value);
  if (/[",¥r
]/.test(v)) {
    return `"${v.replace(/"/g, '""')}"`;
  }
  return v;
}
```

`packages/shared/src/audit/schema.ts` — SAFE_DIRECT_COPY

```
import { z } from 'zod';

export const auditActionSchema = z.enum([
  'auth.login',
  'auth.logout',
  'session.start',
  'session.end',
  'file.transfer.attempted',
  'clipboard.sync.toggled',
  'input.permission.changed',
  'audit.export.created',
  'support.ticket.created',
  'support.ticket.status.changed',
]);

export const auditLogInputSchema = z.object({
  action: auditActionSchema,
  userId: z.string().min(1).nullish(),
  deviceId: z.string().min(1).nullish(),
  metadata: z.record(z.string(), z.string()).optional(),
});

export const complianceExportInputSchema = z.object({
  fromDate: z.string().datetime(),
  toDate: z.string().datetime(),
});

export type AuditLogInputParsed = z.infer<typeof auditLogInputSchema>;
export type ComplianceExportInputParsed = z.infer<typeof complianceExportInputSchema>;
```

`packages/shared/src/audit/index.ts` — SAFE_DIRECT_COPY

```
export * from './types';
export * from './redaction';
export * from './exportCsv';
export * from './schema';
```

Part B — Shared package: support

`packages/shared/src/support/types.ts` — SAFE_DIRECT_COPY

```
import type { TicketStatus } from './status';
import type { TicketPriority } from './priority';

export interface SupportTicketDto {
  id: string;
  orgId: string;
  createdById: string;
  subject: string;
  body: string;
  status: TicketStatus;
  priority: TicketPriority;
  deviceId: string | null;
  createdAt: string;
```

```
  updatedAt: string;
}

export interface SupportTicketCommentDto {
  id: string;
  ticketId: string;
  authorId: string;
  body: string;
  /** Internal notes are not shown to the requester. */
  visibility: 'public' | 'internal';
  createdAt: string;
}

export interface SupportTicketEventDto {
  id: string;
  ticketId: string;
  type: 'created' | 'status_changed' | 'commented';
  detail: Record<string, string>;
  createdAt: string;
}
```

`packages/shared/src/support/status.ts` — SAFE_DIRECT_COPY

```
/** Support ticket lifecycle status. */
export type TicketStatus = 'open' | 'pending' | 'resolved' | 'closed';

export const TICKET_STATUSES: readonly TicketStatus[] = ['open', 'pending', 'resolved', 'closed'];

/** Allowed status transitions. Closed is terminal. */
const TRANSITIONS: Record<TicketStatus, TicketStatus[]> = {
  open: ['pending', 'resolved', 'closed'],
  pending: ['open', 'resolved', 'closed'],
  resolved: ['closed', 'open'],
  closed: [],
};

export function canTransition(from: TicketStatus, to: TicketStatus): boolean {
  return TRANSITIONS[from].includes(to);
}

export function isOpenStatus(status: TicketStatus): boolean {
  return status === 'open' || status === 'pending';
}
```

`packages/shared/src/support/priority.ts` — SAFE_DIRECT_COPY

```
/** Support ticket priority. */
export type TicketPriority = 'low' | 'normal' | 'high' | 'urgent';

export const TICKET_PRIORITIES: readonly TicketPriority[] = ['low', 'normal', 'high', 'urgent'];

const RANK: Record<TicketPriority, number> = { low: 1, normal: 2, high: 3, urgent: 4 };

export function priorityRank(p: TicketPriority): number {
  return RANK[p];
}
```

```
/** Sort comparator: highest priority first. */
export function byPriorityDesc(a: TicketPriority, b: TicketPriority): number {
  return RANK[b] - RANK[a];
}

/** Default priority for a new ticket. */
export const DEFAULT_PRIORITY: TicketPriority = 'normal';
```

`packages/shared/src/support/schema.ts` — SAFE_DIRECT_COPY

```
import { z } from 'zod';
import { TICKET_PRIORITIES } from './priority';
import { TICKET_STATUSES } from './status';

export const createTicketSchema = z.object({
  subject: z.string().min(1).max(200),
  body: z.string().min(1).max(10_000),
  priority: z.enum(TICKET_PRIORITIES as unknown as [string, ...string[]]).optional(),
  deviceId: z.string().min(1).nullish(),
});

export const addCommentSchema = z.object({
  body: z.string().min(1).max(10_000),
  visibility: z.enum(['public', 'internal']).default('public'),
});

export const updateStatusSchema = z.object({
  status: z.enum(TICKET_STATUSES as unknown as [string, ...string[]]),
});

export type CreateTicketInput = z.infer<typeof createTicketSchema>;
export type AddCommentInput = z.infer<typeof addCommentSchema>;
export type UpdateStatusInput = z.infer<typeof updateStatusSchema>;
```

`packages/shared/src/support/index.ts` — SAFE_DIRECT_COPY

```
export * from './types';
export * from './status';
export * from './priority';
export * from './schema';
```

`packages/shared/src/index.ts` — REVIEW_REQUIRED (patch)

```
// ...existing exports...
export * from './audit';
export * from './support';
```

`packages/shared/src/errors/errorCodes.ts` — REVIEW_REQUIRED (patch)

```
export const ErrorCodes = {
  // ...existing codes...
  AUDIT_EXPORT_NOT_FOUND: 'AUDIT_EXPORT_NOT_FOUND',
  SUPPORT_TICKET_NOT_FOUND: 'SUPPORT_TICKET_NOT_FOUND',
  INVALID_STATUS_TRANSITION: 'INVALID_STATUS_TRANSITION',
} as const;
```

Part C — API: Prisma schema + migration

`apps/api/prisma/schema.prisma` — REVIEW_REQUIRED (patch)

> Append these five models. They link to existing `User` and `Device` models where useful. Run `prisma generate` after merging.

```
model AuditLog {
  id        String   @id @default(cuid())
  orgId     String
  action    String
  userId    String?
  deviceId  String?
  /// Redacted, content-light metadata. NEVER stores secrets.
  metadata  Json
  createdAt DateTime @default(now())

  user   User?   @relation(fields: [userId], references: [id], onDelete: SetNull)
  device Device? @relation(fields: [deviceId], references: [id], onDelete: SetNull)

  @@index([orgId])
  @@index([orgId, createdAt])
  @@index([action])
}

model ComplianceExport {
  id        String   @id @default(cuid())
  orgId     String
  fromDate  DateTime
  toDate    DateTime
  rowCount  Int      @default(0)
  status    String   // "ready" | "empty"
  createdById String
  createdAt DateTime @default(now())

  @@index([orgId])
  @@index([orgId, createdAt])
}

model SupportTicket {
  id          String   @id @default(cuid())
  orgId       String
  createdById String
  subject     String
  body        String
  status      String   @default("open")
  priority    String   @default("normal")
  deviceId    String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  device   Device?               @relation(fields: [deviceId], references: [id], onDelete: SetNull)
  comments SupportTicketComment[]
  events   SupportTicketEvent[]

  @@index([orgId])
  @@index([orgId, status])
```

```
}

model SupportTicketComment {
  id         String   @id @default(cuid())
  ticketId   String
  authorId   String
  body       String
  visibility String   @default("public") // "public" | "internal"
  createdAt  DateTime @default(now())

  ticket SupportTicket @relation(fields: [ticketId], references: [id], onDelete: Cascade)

  @@index([ticketId])
}

model SupportTicketEvent {
  id        String   @id @default(cuid())
  ticketId  String
  type      String   // "created" | "status_changed" | "commented"
  detail    Json
  createdAt DateTime @default(now())

  ticket SupportTicket @relation(fields: [ticketId], references: [id], onDelete: Cascade)

  @@index([ticketId])
}
```

> Back-relations on existing models (REVIEW_REQUIRED):
>
> `// model User { ... add: }`
> `auditLogs AuditLog[]`
>
> `// model Device { ... add: }`
> `auditLogs      AuditLog[]`
> `supportTickets SupportTicket[]`

`apps/api/prisma/migrations/0014_audit_support/migration.sql` — SAFE_DIRECT_COPY

```
-- AuditLog
CREATE TABLE "AuditLog" (
  "id"        TEXT PRIMARY KEY,
  "orgId"     TEXT NOT NULL,
  "action"    TEXT NOT NULL,
  "userId"    TEXT REFERENCES "User"("id") ON DELETE SET NULL,
  "deviceId"  TEXT REFERENCES "Device"("id") ON DELETE SET NULL,
  "metadata"  JSONB NOT NULL,
  "createdAt" TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "AuditLog_orgId_idx" ON "AuditLog"("orgId");
CREATE INDEX "AuditLog_orgId_createdAt_idx" ON "AuditLog"("orgId", "createdAt");
CREATE INDEX "AuditLog_action_idx" ON "AuditLog"("action");

-- ComplianceExport
CREATE TABLE "ComplianceExport" (
  "id"          TEXT PRIMARY KEY,
  "orgId"       TEXT NOT NULL,
  "fromDate"    TIMESTAMP NOT NULL,
```

```
  "toDate"      TIMESTAMP NOT NULL,
  "rowCount"    INTEGER NOT NULL DEFAULT 0,
  "status"      TEXT NOT NULL,
  "createdById" TEXT NOT NULL,
  "createdAt"   TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "ComplianceExport_orgId_idx" ON "ComplianceExport"("orgId");
CREATE INDEX "ComplianceExport_orgId_createdAt_idx" ON "ComplianceExport"("orgId", "createdAt");

-- SupportTicket
CREATE TABLE "SupportTicket" (
  "id"          TEXT PRIMARY KEY,
  "orgId"       TEXT NOT NULL,
  "createdById" TEXT NOT NULL,
  "subject"     TEXT NOT NULL,
  "body"        TEXT NOT NULL,
  "status"      TEXT NOT NULL DEFAULT 'open',
  "priority"    TEXT NOT NULL DEFAULT 'normal',
  "deviceId"    TEXT REFERENCES "Device"("id") ON DELETE SET NULL,
  "createdAt"   TIMESTAMP NOT NULL DEFAULT now(),
  "updatedAt"   TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "SupportTicket_orgId_idx" ON "SupportTicket"("orgId");
CREATE INDEX "SupportTicket_orgId_status_idx" ON "SupportTicket"("orgId", "status");

-- SupportTicketComment
CREATE TABLE "SupportTicketComment" (
  "id"         TEXT PRIMARY KEY,
  "ticketId"   TEXT NOT NULL REFERENCES "SupportTicket"("id") ON DELETE CASCADE,
  "authorId"   TEXT NOT NULL,
  "body"       TEXT NOT NULL,
  "visibility" TEXT NOT NULL DEFAULT 'public',
  "createdAt"  TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "SupportTicketComment_ticketId_idx" ON "SupportTicketComment"("ticketId");

-- SupportTicketEvent
CREATE TABLE "SupportTicketEvent" (
  "id"        TEXT PRIMARY KEY,
  "ticketId"  TEXT NOT NULL REFERENCES "SupportTicket"("id") ON DELETE CASCADE,
  "type"      TEXT NOT NULL,
  "detail"    JSONB NOT NULL,
  "createdAt" TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "SupportTicketEvent_ticketId_idx" ON "SupportTicketEvent"("ticketId");
```

Part D — API: lib, repositories, routes

`apps/api/src/lib/auditLogger.ts` — SAFE_DIRECT_COPY

```
import {
  redactMetadata,
  assertRedacted,
  type AuditAction,
```

```
  type AuditLogDto,
} from '@remotedesk/shared';
import { auditRepository } from '../repositories/auditRepository';

/**
 * Append-only audit writer. Redaction is MANDATORY and runs before persistence;
 * `assertRedacted` is a defense-in-depth backstop. No secrets are ever stored.
 */
export const auditLogger = {
  async record(input: {
    orgId: string;
    action: AuditAction;
    userId?: string | null;
    deviceId?: string | null;
    metadata?: Record<string, string>;
  }): Promise<AuditLogDto> {
    const redacted = redactMetadata(input.metadata ?? {});
    assertRedacted(redacted);
    return auditRepository.insert({
      orgId: input.orgId,
      action: input.action,
      userId: input.userId ?? null,
      deviceId: input.deviceId ?? null,
      metadata: redacted,
    });
  },
};
```

`apps/api/src/lib/complianceExports.ts` — SAFE_DIRECT_COPY

```
import { auditLogsToCsv, type AuditLogDto, type ComplianceExportDto } from '@remotedesk/shared';
import { auditRepository } from '../repositories/auditRepository';

/**
 * Builds a compliance CSV from audit logs in a date range. The CSV serializer
 * re-redacts metadata, so an export can never contain secrets. Returns the
 * recorded export job + the CSV body.
 */
export const complianceExports = {
  async create(input: {
    orgId: string;
    createdById: string;
    fromDate: string;
    toDate: string;
  }): Promise<{ export: ComplianceExportDto; csv: string }> {
    const logs: AuditLogDto[] = await auditRepository.listForRange(
      input.orgId,
      new Date(input.fromDate),
      new Date(input.toDate),
    );
    const csv = auditLogsToCsv(logs);
    const job = await auditRepository.recordExport({
      orgId: input.orgId,
      createdById: input.createdById,
      fromDate: new Date(input.fromDate),
      toDate: new Date(input.toDate),
      rowCount: logs.length,
```

```
      status: logs.length > 0 ? 'ready' : 'empty',
    });
    return { export: job, csv };
  },
};
```

`apps/api/src/lib/supportTickets.ts` — SAFE_DIRECT_COPY

```
import {
  AppError,
  ErrorCodes,
  canTransition,
  DEFAULT_PRIORITY,
  type SupportTicketDto,
  type SupportTicketCommentDto,
  type TicketPriority,
  type TicketStatus,
} from '@remotedesk/shared';
import { supportRepository } from '../repositories/supportRepository';

export const supportTickets = {
  create: (input: {
    orgId: string;
    createdById: string;
    subject: string;
    body: string;
    priority?: TicketPriority;
    deviceId?: string | null;
  }): Promise<SupportTicketDto> =>
    supportRepository.createTicket({
      ...input,
      priority: input.priority ?? DEFAULT_PRIORITY,
      deviceId: input.deviceId ?? null,
    }),

  list: (orgId: string) => supportRepository.listTickets(orgId),

  async get(orgId: string, ticketId: string): Promise<{
    ticket: SupportTicketDto;
    comments: SupportTicketCommentDto[];
  }> {
    const ticket = await supportRepository.getTicket(orgId, ticketId);
    if (!ticket) throw new AppError(ErrorCodes.SUPPORT_TICKET_NOT_FOUND);
    const comments = await supportRepository.listComments(ticketId);
    return { ticket, comments };
  },

  async comment(input: {
    orgId: string;
    ticketId: string;
    authorId: string;
    body: string;
    visibility: 'public' | 'internal';
  }): Promise<SupportTicketCommentDto> {
    const ticket = await supportRepository.getTicket(input.orgId, input.ticketId);
    if (!ticket) throw new AppError(ErrorCodes.SUPPORT_TICKET_NOT_FOUND);
    return supportRepository.addComment(input);
```

```
  },

  async setStatus(input: {
    orgId: string;
    ticketId: string;
    status: TicketStatus;
  }): Promise<SupportTicketDto> {
    const ticket = await supportRepository.getTicket(input.orgId, input.ticketId);
    if (!ticket) throw new AppError(ErrorCodes.SUPPORT_TICKET_NOT_FOUND);
    if (!canTransition(ticket.status, input.status)) {
      throw new AppError(ErrorCodes.INVALID_STATUS_TRANSITION);
    }
    return supportRepository.updateStatus(input.ticketId, input.status, ticket.status);
  },
};
```

`apps/api/src/repositories/auditRepository.ts` — SAFE_DIRECT_COPY

```
import type { AuditLogDto, ComplianceExportDto } from '@remotedesk/shared';
import { prisma } from '../db/prisma';

function logToDto(l: any): AuditLogDto {
  return {
    id: l.id,
    orgId: l.orgId,
    action: l.action,
    actor: { userId: l.userId ?? null, deviceId: l.deviceId ?? null },
    metadata: (l.metadata as Record<string, string>) ?? {},
    createdAt: l.createdAt.toISOString(),
  };
}

export const auditRepository = {
  async insert(input: {
    orgId: string;
    action: string;
    userId: string | null;
    deviceId: string | null;
    metadata: Record<string, string>;
  }): Promise<AuditLogDto> {
    const l = await prisma.auditLog.create({
      data: {
        orgId: input.orgId,
        action: input.action,
        userId: input.userId,
        deviceId: input.deviceId,
        metadata: input.metadata as unknown as object,
      },
    });
    return logToDto(l);
  },

  async list(orgId: string, limit = 200): Promise<AuditLogDto[]> {
    const rows = await prisma.auditLog.findMany({
      where: { orgId },
      orderBy: { createdAt: 'desc' },
      take: limit,
```

```
      });
      return rows.map(logToDto);
    },

    async listForRange(orgId: string, from: Date, to: Date): Promise<AuditLogDto[]> {
      const rows = await prisma.auditLog.findMany({
        where: { orgId, createdAt: { gte: from, lte: to } },
        orderBy: { createdAt: 'asc' },
      });
      return rows.map(logToDto);
    },

    async recordExport(input: {
      orgId: string;
      createdById: string;
      fromDate: Date;
      toDate: Date;
      rowCount: number;
      status: 'ready' | 'empty';
    }): Promise<ComplianceExportDto> {
      const e = await prisma.complianceExport.create({ data: input });
      return {
        id: e.id,
        orgId: e.orgId,
        fromDate: e.fromDate.toISOString(),
        toDate: e.toDate.toISOString(),
        rowCount: e.rowCount,
        status: e.status as 'ready' | 'empty',
        createdAt: e.createdAt.toISOString(),
      };
    },

    async exportById(orgId: string, id: string): Promise<ComplianceExportDto | null> {
      const e = await prisma.complianceExport.findFirst({ where: { id, orgId } });
      if (!e) return null;
      return {
        id: e.id,
        orgId: e.orgId,
        fromDate: e.fromDate.toISOString(),
        toDate: e.toDate.toISOString(),
        rowCount: e.rowCount,
        status: e.status as 'ready' | 'empty',
        createdAt: e.createdAt.toISOString(),
      };
    },
  };
```

`apps/api/src/repositories/supportRepository.ts` — SAFE_DIRECT_COPY

```
  import type {
    SupportTicketDto,
    SupportTicketCommentDto,
    TicketPriority,
    TicketStatus,
  } from '@remotedesk/shared';
  import { prisma } from '../db/prisma';
```

```
function ticketToDto(t: any): SupportTicketDto {
  return {
    id: t.id,
    orgId: t.orgId,
    createdById: t.createdById,
    subject: t.subject,
    body: t.body,
    status: t.status as TicketStatus,
    priority: t.priority as TicketPriority,
    deviceId: t.deviceId ?? null,
    createdAt: t.createdAt.toISOString(),
    updatedAt: t.updatedAt.toISOString(),
  };
}

function commentToDto(c: any): SupportTicketCommentDto {
  return {
    id: c.id,
    ticketId: c.ticketId,
    authorId: c.authorId,
    body: c.body,
    visibility: c.visibility as 'public' | 'internal',
    createdAt: c.createdAt.toISOString(),
  };
}

export const supportRepository = {
  async createTicket(input: {
    orgId: string;
    createdById: string;
    subject: string;
    body: string;
    priority: TicketPriority;
    deviceId: string | null;
  }): Promise<SupportTicketDto> {
    const t = await prisma.supportTicket.create({ data: input });
    await prisma.supportTicketEvent.create({
      data: { ticketId: t.id, type: 'created', detail: { priority: input.priority } as object },
    });
    return ticketToDto(t);
  },

  async listTickets(orgId: string): Promise<SupportTicketDto[]> {
    const rows = await prisma.supportTicket.findMany({
      where: { orgId },
      orderBy: { updatedAt: 'desc' },
    });
    return rows.map(ticketToDto);
  },

  async getTicket(orgId: string, ticketId: string): Promise<SupportTicketDto | null> {
    const t = await prisma.supportTicket.findFirst({ where: { id: ticketId, orgId } });
    return t ? ticketToDto(t) : null;
  },

  async listComments(ticketId: string): Promise<SupportTicketCommentDto[]> {
    const rows = await prisma.supportTicketComment.findMany({
```

```
      where: { ticketId },
      orderBy: { createdAt: 'asc' },
    });
    return rows.map(commentToDto);
  },

  async addComment(input: {
    ticketId: string;
    authorId: string;
    body: string;
    visibility: 'public' | 'internal';
  }): Promise<SupportTicketCommentDto> {
    const c = await prisma.supportTicketComment.create({ data: input });
    await prisma.supportTicketEvent.create({
      data: { ticketId: input.ticketId, type: 'commented', detail: { visibility: input.visibility } as object },
    });
    return commentToDto(c);
  },

  async updateStatus(
    ticketId: string,
    status: TicketStatus,
    fromStatus: TicketStatus,
  ): Promise<SupportTicketDto> {
    const t = await prisma.supportTicket.update({ where: { id: ticketId }, data: { status } });
    await prisma.supportTicketEvent.create({
      data: { ticketId, type: 'status_changed', detail: { from: fromStatus, to: status } as object },
    });
    return ticketToDto(t);
  },
};
```

`apps/api/src/routes/audit.routes.ts` — SAFE_DIRECT_COPY

```
import { Router } from 'express';
import {
  auditLogInputSchema,
  complianceExportInputSchema,
  AppError,
  ErrorCodes,
} from '@remotedesk/shared';
import { requireAuth, type AuthedRequest } from '../middleware/requireAuth.middleware';
import { orgContext, type OrgRequest } from '../middleware/orgContext.middleware';
import { auditLogger } from '../lib/auditLogger';
import { auditRepository } from '../repositories/auditRepository';
import { complianceExports } from '../lib/complianceExports';

const h =
  (fn: (req: any, res: any) => Promise<void>) =>
  (req: any, res: any, next: any) =>
    fn(req, res).catch(next);

/**
 * /api/audit
 *   GET  /logs                 -> recent audit logs (redacted)
 *   POST /logs                 -> append an audit log (metadata redacted server-side)
 *   POST /exports              -> create a compliance CSV export (returns CSV body)
```

```
 *   GET  /exports/:exportId      -> fetch export job metadata
 */
export const auditRoutes = Router()
  .get('/logs', requireAuth, orgContext, h(async (req, res) => {
    res.json(await auditRepository.list((req as OrgRequest).orgId));
  }))
  .post('/logs', requireAuth, orgContext, h(async (req, res) => {
    const input = auditLogInputSchema.parse(req.body);
    const r = req as OrgRequest & AuthedRequest;
    res.status(201).json(await auditLogger.record({ orgId: r.orgId, ...input }));
  }))
  .post('/exports', requireAuth, orgContext, h(async (req, res) => {
    const { fromDate, toDate } = complianceExportInputSchema.parse(req.body);
    const r = req as OrgRequest & AuthedRequest;
    const { export: job, csv } = await complianceExports.create({
      orgId: r.orgId, createdById: r.userId, fromDate, toDate,
    });
    res.setHeader('content-type', 'text/csv');
    res.setHeader('content-disposition', `attachment; filename="audit-export-${job.id}.csv"`);
    res.setHeader('x-export-id', job.id);
    res.setHeader('x-export-rows', String(job.rowCount));
    res.status(201).send(csv);
  }))
  .get('/exports/:exportId', requireAuth, orgContext, h(async (req, res) => {
    const job = await auditRepository.exportById((req as OrgRequest).orgId, req.params.exportId);
    if (!job) throw new AppError(ErrorCodes.AUDIT_EXPORT_NOT_FOUND);
    res.json(job);
  }));
```

`apps/api/src/routes/support.routes.ts` — SAFE_DIRECT_COPY

```
import { Router } from 'express';
import {
  createTicketSchema,
  addCommentSchema,
  updateStatusSchema,
  type TicketPriority,
  type TicketStatus,
} from '@remotedesk/shared';
import { requireAuth, type AuthedRequest } from '../middleware/requireAuth.middleware';
import { orgContext, type OrgRequest } from '../middleware/orgContext.middleware';
import { supportTickets } from '../lib/supportTickets';

const h =
  (fn: (req: any, res: any) => Promise<void>) =>
  (req: any, res: any, next: any) =>
    fn(req, res).catch(next);

/**
 * /api/support
 *   GET  /tickets                     -> list org tickets
 *   POST /tickets                     -> create a ticket
 *   GET  /tickets/:ticketId           -> ticket + comments
 *   POST /tickets/:ticketId/comments  -> add comment (public|internal)
 *   PATCH /tickets/:ticketId/status   -> change status (validated transition)
 */
export const supportRoutes = Router()
```

```
  .get('/tickets', requireAuth, orgContext, h(async (req, res) => {
    res.json(await supportTickets.list((req as OrgRequest).orgId));
  }))
  .post('/tickets', requireAuth, orgContext, h(async (req, res) => {
    const input = createTicketSchema.parse(req.body);
    const r = req as OrgRequest & AuthedRequest;
    res.status(201).json(await supportTickets.create({
      orgId: r.orgId,
      createdById: r.userId,
      subject: input.subject,
      body: input.body,
      priority: input.priority as TicketPriority | undefined,
      deviceId: input.deviceId ?? null,
    }));
  }))
  .get('/tickets/:ticketId', requireAuth, orgContext, h(async (req, res) => {
    res.json(await supportTickets.get((req as OrgRequest).orgId, req.params.ticketId));
  }))
  .post('/tickets/:ticketId/comments', requireAuth, orgContext, h(async (req, res) => {
    const input = addCommentSchema.parse(req.body);
    const r = req as OrgRequest & AuthedRequest;
    res.status(201).json(await supportTickets.comment({
      orgId: r.orgId,
      ticketId: req.params.ticketId,
      authorId: r.userId,
      body: input.body,
      visibility: input.visibility,
    }));
  }))
  .patch('/tickets/:ticketId/status', requireAuth, orgContext, h(async (req, res) => {
    const { status } = updateStatusSchema.parse(req.body);
    res.json(await supportTickets.setStatus({
      orgId: (req as OrgRequest).orgId,
      ticketId: req.params.ticketId,
      status: status as TicketStatus,
    }));
  }));
```

`apps/api/src/server.ts` — REVIEW_REQUIRED (patch)

```
// ...existing imports...
import { auditRoutes } from './routes/audit.routes';
import { supportRoutes } from './routes/support.routes';

// ...inside createServer(), next to the other app.use('/api/...') mounts:
app.use('/api/audit', auditRoutes);
app.use('/api/support', supportRoutes);
```

Part E — Web: audit + support pages

`apps/web/app/dashboard/audit/auditApi.ts` — SAFE_DIRECT_COPY

```
import type { AuditLogDto } from '@remotedesk/shared';
import { api } from '../../../src/api/apiClient';
```

```
export const auditApi = {
  logs: () => api.get<AuditLogDto[]>('/audit/logs'),
  /** Returns the CSV body as text + the export id header. */
  async export(fromDate: string, toDate: string): Promise<{ csv: string; exportId: string | null }> {
    const res = await api.raw('/audit/exports', {
      method: 'POST',
      body: JSON.stringify({ fromDate, toDate }),
      headers: { 'content-type': 'application/json' },
    });
    const csv = await res.text();
    return { csv, exportId: res.headers.get('x-export-id') };
  },
};
```

> Note: assumes `apiClient` exposes a `raw()` passthrough returning a `Response`.
> If not, add one or inline a `fetch` with the auth header. (REVIEW in code review.)

`apps/web/app/dashboard/audit/page.tsx` — SAFE_DIRECT_COPY

```
'use client';

import React, { useEffect, useState } from 'react';
import type { AuditLogDto } from '@remotedesk/shared';
import { auditApi } from './auditApi';
import styles from '../support/support.module.css';

export default function AuditPage() {
  const [logs, setLogs] = useState<AuditLogDto[]>([]);
  const [from, setFrom] = useState('');
  const [to, setTo] = useState('');
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    auditApi.logs().then(setLogs).catch(() => setError('Could not load audit logs.'));
  }, []);

  const runExport = async () => {
    if (!from || !to) { setError('Pick a date range first.'); return; }
    setBusy(true);
    setError(null);
    try {
      const { csv, exportId } = await auditApi.export(
        new Date(from).toISOString(),
        new Date(to).toISOString(),
      );
      const blob = new Blob([csv], { type: 'text/csv' });
      const url = URL.createObjectURL(blob);
      const a = document.createElement('a');
      a.href = url;
      a.download = `audit-export-${exportId ?? 'range'}.csv`;
      a.click();
      URL.revokeObjectURL(url);
    } catch {
      setError('Export failed.');
    } finally {
      setBusy(false);
```

```
    }
  };

  return (
    <main className={styles.page}>
      <header className={styles.header}>
        <h1>Audit log</h1>
        <p className={styles.sub}>Redacted, append-only. Export a date range to CSV for compliance.</p>
      </header>

      {error && <div className={styles.error}>{error}</div>}

      <section className={styles.panel}>
        <h2>Export</h2>
        <div className={styles.row}>
          <label className={styles.field}><span>From</span><input type="date" value={from} onChange={(e) =>
setFrom(e.target.value)} /></label>
          <label className={styles.field}><span>To</span><input type="date" value={to} onChange={(e) =>
setTo(e.target.value)} /></label>
          <button type="button" className={styles.btnPrimary} disabled={busy} onClick={runExport}>
            {busy ? 'Exporting…' : 'Export CSV'}
          </button>
        </div>
      </section>

      <section className={styles.panel}>
        <h2>Recent events</h2>
        <table className={styles.table}>
          <thead><tr><th>Time</th><th>Action</th><th>User</th><th>Device</th></tr></thead>
          <tbody>
            {logs.map((l) => (
              <tr key={l.id}>
                <td>{new Date(l.createdAt).toLocaleString()}</td>
                <td>{l.action}</td>
                <td>{l.actor.userId ?? '—'}</td>
                <td>{l.actor.deviceId ?? '—'}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </section>
    </main>
  );
}
```

`apps/web/app/dashboard/support/supportApi.ts` — SAFE_DIRECT_COPY

```
import type {
  SupportTicketDto,
  SupportTicketCommentDto,
  TicketPriority,
  TicketStatus,
} from '@remotedesk/shared';
import { api } from '../../../src/api/apiClient';

export const supportApi = {
  list: () => api.get<SupportTicketDto[]>('/support/tickets'),
```

```
  create: (input: { subject: string; body: string; priority?: TicketPriority }) =>
    api.post<SupportTicketDto>('/support/tickets', input),
  get: (ticketId: string) =>
    api.get<{ ticket: SupportTicketDto; comments: SupportTicketCommentDto[] }>(`/support/tickets/${ticketId}`),
  comment: (ticketId: string, body: string, visibility: 'public' | 'internal') =>
    api.post<SupportTicketCommentDto>(`/support/tickets/${ticketId}/comments`, { body, visibility }),
  setStatus: (ticketId: string, status: TicketStatus) =>
    api.patch<SupportTicketDto>(`/support/tickets/${ticketId}/status`, { status }),
};
```

`apps/web/app/dashboard/support/page.tsx` — SAFE_DIRECT_COPY

```
'use client';

import React, { useEffect, useState } from 'react';
import Link from 'next/link';
import type { SupportTicketDto, TicketPriority } from '@remotedesk/shared';
import { TICKET_PRIORITIES } from '@remotedesk/shared';
import { supportApi } from './supportApi';
import styles from './support.module.css';

export default function SupportPage() {
  const [tickets, setTickets] = useState<SupportTicketDto[]>([]);
  const [subject, setSubject] = useState('');
  const [body, setBody] = useState('');
  const [priority, setPriority] = useState<TicketPriority>('normal');
  const [error, setError] = useState<string | null>(null);

  const load = () => supportApi.list().then(setTickets).catch(() => setError('Could not load tickets.'));
  useEffect(() => { void load(); }, []);

  const create = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!subject || !body) return;
    try {
      await supportApi.create({ subject, body, priority });
      setSubject(''); setBody(''); setPriority('normal');
      await load();
    } catch {
      setError('Could not create ticket.');
    }
  };

  return (
    <main className={styles.page}>
      <header className={styles.header}><h1>Support</h1><p className={styles.sub}>Create and track support tickets.</p>
</header>
      {error && <div className={styles.error}>{error}</div>}

      <section className={styles.panel}>
        <h2>New ticket</h2>
        <form onSubmit={create} className={styles.form}>
          <input placeholder="Subject" value={subject} onChange={(e) => setSubject(e.target.value)} />
          <textarea placeholder="Describe the issue" value={body} onChange={(e) => setBody(e.target.value)} />
          <div className={styles.row}>
            <select value={priority} onChange={(e) => setPriority(e.target.value as TicketPriority)}>
              {TICKET_PRIORITIES.map((p) => <option key={p} value={p}>{p}</option>)}
```

```
            </select>
            <button type="submit" className={styles.btnPrimary}>Create ticket</button>
          </div>
        </form>
      </section>

      <section className={styles.panel}>
        <h2>Tickets</h2>
        <table className={styles.table}>
          <thead><tr><th>Subject</th><th>Status</th><th>Priority</th><th>Updated</th></tr></thead>
          <tbody>
            {tickets.map((t) => (
              <tr key={t.id}>
                <td><Link href={`/dashboard/support/${t.id}`}>{t.subject}</Link></td>
                <td><span className={styles.badge} data-status={t.status}>{t.status}</span></td>
                <td>{t.priority}</td>
                <td>{new Date(t.updatedAt).toLocaleString()}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </section>
    </main>
  );
}
```

`apps/web/app/dashboard/support/[ticketId]/page.tsx` — SAFE_DIRECT_COPY

```
'use client';

import React, { useCallback, useEffect, useState } from 'react';
import { useParams } from 'next/navigation';
import type { SupportTicketDto, SupportTicketCommentDto, TicketStatus } from '@remotedesk/shared';
import { TICKET_STATUSES, canTransition } from '@remotedesk/shared';
import { supportApi } from '../supportApi';
import styles from '../support.module.css';

export default function TicketDetailPage() {
  const { ticketId } = useParams<{ ticketId: string }>();
  const [ticket, setTicket] = useState<SupportTicketDto | null>(null);
  const [comments, setComments] = useState<SupportTicketCommentDto[]>([]);
  const [body, setBody] = useState('');
  const [visibility, setVisibility] = useState<'public' | 'internal'>('public');
  const [error, setError] = useState<string | null>(null);

  const load = useCallback(async () => {
    try {
      const data = await supportApi.get(ticketId);
      setTicket(data.ticket);
      setComments(data.comments);
    } catch {
      setError('Could not load ticket.');
    }
  }, [ticketId]);

  useEffect(() => { void load(); }, [load]);
```

```
  const addComment = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!body) return;
    await supportApi.comment(ticketId, body, visibility);
    setBody('');
    await load();
  };

  const changeStatus = async (status: TicketStatus) => {
    try {
      const updated = await supportApi.setStatus(ticketId, status);
      setTicket(updated);
    } catch {
      setError('That status change is not allowed.');
    }
  };

  if (error) return <main className={styles.page}><div className={styles.error}>{error}</div></main>;
  if (!ticket) return <main className={styles.page}><p className={styles.muted}>Loading…</p></main>;

  return (
    <main className={styles.page}>
      <header className={styles.header}>
        <h1>{ticket.subject}</h1>
        <p className={styles.sub}>
          <span className={styles.badge} data-status={ticket.status}>{ticket.status}</span> · {ticket.priority}
        </p>
      </header>

      <section className={styles.panel}>
        <p>{ticket.body}</p>
        <div className={styles.row}>
          {TICKET_STATUSES.filter((s) => canTransition(ticket.status, s)).map((s) => (
            <button key={s} type="button" className={styles.btnSecondary} onClick={() => changeStatus(s)}>
              Mark {s}
            </button>
          ))}
        </div>
      </section>

      <section className={styles.panel}>
        <h2>Comments</h2>
        {comments.length === 0 && <p className={styles.muted}>No comments yet.</p>}
        {comments.map((c) => (
          <div key={c.id} className={styles.comment} data-visibility={c.visibility}>
            <div className={styles.commentMeta}>
              {c.authorId} · {new Date(c.createdAt).toLocaleString()}
              {c.visibility === 'internal' && <span className={styles.internalTag}>internal</span>}
            </div>
            <p>{c.body}</p>
          </div>
        ))}
        <form onSubmit={addComment} className={styles.form}>
          <textarea placeholder="Add a comment" value={body} onChange={(e) => setBody(e.target.value)} />
          <div className={styles.row}>
            <select value={visibility} onChange={(e) => setVisibility(e.target.value as 'public' | 'internal')}>
              <option value="public">public reply</option>
```

```
              <option value="internal">internal note</option>
            </select>
            <button type="submit" className={styles.btnPrimary}>Add comment</button>
          </div>
        </form>
      </section>
    </main>
  );
}
```

`apps/web/app/dashboard/support/support.module.css` — SAFE_DIRECT_COPY

```
.page { padding: 24px; max-width: 1000px; margin: 0 auto; }
.header h1 { margin: 0 0 4px; font-size: 22px; }
.sub { color: var(--muted, #6b7280); margin: 0 0 20px; }
.error { background: #fef2f2; color: #991b1b; padding: 10px 14px; border-radius: 8px; margin-bottom: 16px; }
.muted { color: var(--muted, #6b7280); font-size: 13px; }
.panel { border: 1px solid var(--border, #e5e7eb); border-radius: 12px; padding: 16px 18px; margin-bottom: 16px;
background: #fff; }
.panel h2 { font-size: 15px; margin: 0 0 12px; }
.form { display: flex; flex-direction: column; gap: 10px; }
.form input, .form textarea, .form select { padding: 8px 10px; border: 1px solid var(--border, #e5e7eb); border-radius:
8px; font: inherit; }
.form textarea { min-height: 90px; resize: vertical; }
.row { display: flex; align-items: center; gap: 10px; flex-wrap: wrap; }
.field { display: flex; flex-direction: column; gap: 4px; font-size: 13px; }
.table { width: 100%; border-collapse: collapse; font-size: 13px; }
.table th, .table td { text-align: left; padding: 8px 10px; border-bottom: 1px solid var(--border, #e5e7eb); }
.table th { color: var(--muted, #6b7280); font-weight: 600; }
.btnPrimary { background: #4f46e5; color: #fff; border: none; padding: 8px 14px; border-radius: 8px; cursor: pointer; }
.btnSecondary { background: #fff; color: #4f46e5; border: 1px solid #c7d2fe; padding: 6px 12px; border-radius: 8px;
cursor: pointer; }
.badge { font-size: 12px; padding: 2px 10px; border-radius: 999px; background: #eff6ff; color: #1d4ed8; }
.badge[data-status='resolved'] { background: #ecfdf5; color: #065f46; }
.badge[data-status='closed'] { background: #f3f4f6; color: #6b7280; }
.badge[data-status='pending'] { background: #fffbeb; color: #92400e; }
.comment { border-left: 3px solid #e5e7eb; padding: 6px 12px; margin-bottom: 10px; }
.comment[data-visibility='internal'] { border-left-color: #f59e0b; background: #fffbeb; }
.commentMeta { font-size: 12px; color: var(--muted, #6b7280); display: flex; gap: 8px; align-items: center; }
.internalTag { font-size: 11px; background: #f59e0b; color: #fff; padding: 1px 6px; border-radius: 6px; }
```

`apps/web/app/dashboard/page.tsx` — REVIEW_REQUIRED (patch)

```
// ...existing imports...
import Link from 'next/link';

// ...inside the dashboard grid:
<Link href="/dashboard/audit" className="dashboard-card">
  <h3>Audit log</h3>
  <p>Redacted event history and compliance CSV export.</p>
</Link>
<Link href="/dashboard/support" className="dashboard-card">
  <h3>Support</h3>
  <p>Create and track support tickets.</p>
</Link>
```

Part F — Desktop: audit + support clients

`apps/desktop/src/renderer/src/services/auditClient.ts` — SAFE_DIRECT_COPY

```
import { redactMetadata, type AuditAction } from '@remotedesk/shared';

/**
 * Posts critical-event audit logs to the API. FAILS SILENTLY: any error is
 * swallowed so audit reporting never interrupts the session. Metadata is
 * redacted locally before send (defense in depth — server redacts again).
 */
export class AuditClient {
  constructor(
    private readonly baseUrl: string,
    private readonly authToken: string,
  ) {}

  async record(
    action: AuditAction,
    metadata: Record<string, string> = {},
    ids: { userId?: string | null; deviceId?: string | null } = {},
  ): Promise<void> {
    try {
      await fetch(`${this.baseUrl}/api/audit/logs`, {
        method: 'POST',
        headers: {
          'content-type': 'application/json',
          authorization: `Bearer ${this.authToken}`,
        },
        body: JSON.stringify({
          action,
          userId: ids.userId ?? null,
          deviceId: ids.deviceId ?? null,
          metadata: redactMetadata(metadata),
        }),
      });
    } catch {
      // Swallow: audit must never break the session.
    }
  }
}
```

`apps/desktop/src/renderer/src/services/supportClient.ts` — SAFE_DIRECT_COPY

```
import type { SupportTicketDto, TicketPriority } from '@remotedesk/shared';

/** Minimal desktop support-ticket client (create + list). */
export class SupportClient {
  constructor(
    private readonly baseUrl: string,
    private readonly authToken: string,
  ) {}

  private headers() {
    return {
      'content-type': 'application/json',
      authorization: `Bearer ${this.authToken}`,
```

```
      };
    }

    async create(input: {
      subject: string;
      body: string;
      priority?: TicketPriority;
      deviceId?: string | null;
    }): Promise<SupportTicketDto> {
      const res = await fetch(`${this.baseUrl}/api/support/tickets`, {
        method: 'POST',
        headers: this.headers(),
        body: JSON.stringify(input),
      });
      if (!res.ok) throw new Error(`create ticket failed: ${res.status}`);
      return res.json();
    }

    async list(): Promise<SupportTicketDto[]> {
      const res = await fetch(`${this.baseUrl}/api/support/tickets`, { headers: this.headers() });
      if (!res.ok) throw new Error(`list tickets failed: ${res.status}`);
      return res.json();
    }
  }
```

`apps/desktop/src/renderer/src/session/criticalEvents.ts` — REVIEW_REQUIRED (patch)

> Call the audit client from the existing critical-event hooks. **Native input execution stays disabled — this only records that a permission/toggle changed.** Never pass secrets/device passwords in metadata; the client + server redact, but don't rely on it.

```
// ...existing imports...
import { AuditClient } from '../services/auditClient';

// --- construct once when the session/app boots ---
const auditClient = new AuditClient(apiBase, authToken);

// --- wire into existing hooks (fail silently) ---
onLogin((user) => void auditClient.record('auth.login', { method: 'password' }, { userId: user.id }));
onSessionStart((s) => void auditClient.record('session.start', { sessionId: s.id }, { userId: s.userId, deviceId:
s.deviceId }));
onSessionEnd((s) => void auditClient.record('session.end', { sessionId: s.id, reason: s.reason }, { userId: s.userId,
deviceId: s.deviceId }));
onFileTransferAttempt((s, f) => void auditClient.record('file.transfer.attempted', { sessionId: s.id, fileName: f.name,
bytes: String(f.size) }, { deviceId: s.deviceId }));
onClipboardToggle((s, enabled) => void auditClient.record('clipboard.sync.toggled', { sessionId: s.id, enabled:
String(enabled) }, { deviceId: s.deviceId }));
onInputPermissionChange((s, allowed) => void auditClient.record('input.permission.changed', { sessionId: s.id, allowed:
String(allowed) }, { deviceId: s.deviceId }));
```

Part G — Tests

`tests/audit/redaction.test.ts` — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { redactMetadata, assertRedacted, REDACTION_MASK } from '@remotedesk/shared';

describe('audit redaction', () => {
  it('redacts auth headers, cookies, passwords, tokens, api keys, device passwords', () => {
    const out = redactMetadata({
      Authorization: 'Bearer x',
      Cookie: 'sid=1',
      password: 'p',
      devicePassword: 'dp',
      token: 't',
      api_key: 'k',
      note: 'keep me',
    });
    expect(out.Authorization).toBe(REDACTION_MASK);
    expect(out.Cookie).toBe(REDACTION_MASK);
    expect(out.password).toBe(REDACTION_MASK);
    expect(out.devicePassword).toBe(REDACTION_MASK);
    expect(out.token).toBe(REDACTION_MASK);
    expect(out.api_key).toBe(REDACTION_MASK);
    expect(out.note).toBe('keep me');
  });

  it('assertRedacted throws on a surviving secret', () => {
    expect(() => assertRedacted({ token: 'raw' })).toThrow();
    expect(() => assertRedacted({ token: REDACTION_MASK })).not.toThrow();
  });
});
```

`tests/audit/exportCsv.test.ts` — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { auditLogsToCsv, csvQuote, type AuditLogDto } from '@remotedesk/shared';

const row: AuditLogDto = {
  id: 'a1',
  orgId: 'o1',
  action: 'session.start',
  actor: { userId: 'u1', deviceId: 'd1' },
  metadata: { sessionId: 's1', token: 'SHOULD_NOT_APPEAR' },
  createdAt: '2026-06-15T10:00:00.000Z',
};

describe('auditLogsToCsv', () => {
  it('emits a header + one row per log', () => {
    const csv = auditLogsToCsv([row]);
    const lines = csv.split('¥r
');
    expect(lines[0]).toBe('id,createdAt,action,userId,deviceId,metadata');
    expect(lines).toHaveLength(2);
  });

  it('re-redacts metadata so secrets never reach the CSV', () => {
    const csv = auditLogsToCsv([row]);
    expect(csv).not.toContain('SHOULD_NOT_APPEAR');
    expect(csv).toContain('[REDACTED]');
  });
```

```
  it('RFC-4180 quotes values with commas/quotes/newlines', () => {
    expect(csvQuote('a,b')).toBe('"a,b"');
    expect(csvQuote('he said "hi"')).toBe('"he said ""hi"""');
    expect(csvQuote('plain')).toBe('plain');
  });
});
```

`tests/audit/audit-contract.test.ts` — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { auditLogInputSchema, complianceExportInputSchema } from '@remotedesk/shared';

describe('audit API contract', () => {
  it('accepts a valid audit log input', () => {
    const parsed = auditLogInputSchema.parse({
      action: 'file.transfer.attempted',
      deviceId: 'd1',
      metadata: { fileName: 'a.txt' },
    });
    expect(parsed.action).toBe('file.transfer.attempted');
  });

  it('rejects an unknown action', () => {
    expect(() => auditLogInputSchema.parse({ action: 'nope' })).toThrow();
  });

  it('requires ISO datetimes for export range', () => {
    expect(() => complianceExportInputSchema.parse({ fromDate: 'yesterday', toDate: 'today' })).toThrow();
    const ok = complianceExportInputSchema.parse({
      fromDate: '2026-06-01T00:00:00.000Z',
      toDate: '2026-06-30T00:00:00.000Z',
    });
    expect(ok.fromDate).toContain('2026-06-01');
  });
});
```

`tests/support/status.test.ts` — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { canTransition, isOpenStatus, TICKET_STATUSES } from '@remotedesk/shared';

describe('ticket status', () => {
  it('allows open -> resolved -> closed', () => {
    expect(canTransition('open', 'resolved')).toBe(true);
    expect(canTransition('resolved', 'closed')).toBe(true);
  });

  it('treats closed as terminal', () => {
    for (const s of TICKET_STATUSES) {
      expect(canTransition('closed', s)).toBe(false);
    }
  });

  it('classifies open + pending as open statuses', () => {
    expect(isOpenStatus('open')).toBe(true);
    expect(isOpenStatus('pending')).toBe(true);
```

```
    expect(isOpenStatus('closed')).toBe(false);
  });
});
```

`tests/support/priority.test.ts` — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { priorityRank, byPriorityDesc, DEFAULT_PRIORITY, TICKET_PRIORITIES } from '@remotedesk/shared';

describe('ticket priority', () => {
  it('ranks urgent above low', () => {
    expect(priorityRank('urgent')).toBeGreaterThan(priorityRank('low'));
  });

  it('sorts highest priority first', () => {
    const sorted = [...TICKET_PRIORITIES].sort(byPriorityDesc);
    expect(sorted[0]).toBe('urgent');
    expect(sorted[sorted.length - 1]).toBe('low');
  });

  it('defaults to normal', () => {
    expect(DEFAULT_PRIORITY).toBe('normal');
  });
});
```

`tests/support/support-ticket-contract.test.ts` — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { createTicketSchema, addCommentSchema, updateStatusSchema } from '@remotedesk/shared';

describe('support ticket contract', () => {
  it('accepts a valid ticket', () => {
    const t = createTicketSchema.parse({ subject: 'Cannot connect', body: 'Details here' });
    expect(t.subject).toBe('Cannot connect');
  });

  it('rejects an empty subject', () => {
    expect(() => createTicketSchema.parse({ subject: '', body: 'x' })).toThrow();
  });

  it('defaults comment visibility to public', () => {
    const c = addCommentSchema.parse({ body: 'hello' });
    expect(c.visibility).toBe('public');
  });

  it('accepts internal comment visibility', () => {
    const c = addCommentSchema.parse({ body: 'note', visibility: 'internal' });
    expect(c.visibility).toBe('internal');
  });

  it('validates status enum', () => {
    expect(() => updateStatusSchema.parse({ status: 'banana' })).toThrow();
    expect(updateStatusSchema.parse({ status: 'resolved' }).status).toBe('resolved');
  });
});
```

Part H — Docs

`docs/audit-compliance-support.md` — SAFE_DIRECT_COPY

```
# API Audit Logging, Compliance Exports & Support Tickets

Append-only audit logging with mandatory redaction, secret-free compliance CSV
exports, and persisted support tickets. **No live email sending. No fake admin/RBAC.
Native input execution remains disabled.**

## Audit logging
`auditLogger.record()` redacts metadata with the shared `redactMetadata` and runs
`assertRedacted` as a backstop BEFORE persistence. Redacted keys include
Authorization, cookies, tokens, passwords, API keys, and **device passwords**. The
desktop `AuditClient` redacts locally too and **fails silently** so audit never
breaks a session. Critical events wired: login, session start/end, file transfer
attempted, clipboard sync toggled, remote input permission changed.

## Compliance exports
`POST /api/audit/exports` builds a CSV over a date range. The CSV serializer
(`auditLogsToCsv`) **re-redacts** metadata, so an export can never contain secrets
even if a raw row slipped through. Values are RFC-4180 quoted. The job is recorded
in `ComplianceExport` with a row count + status (ready/empty).

## Support tickets
Tickets carry status (open/pending/resolved/closed) and priority (low/normal/high/
urgent). Status changes are validated against an allow-list of transitions
(`canTransition`); closed is terminal. Comments distinguish **public** replies from
**internal** notes, and internal notes are visually separated in the UI. Every
create / comment / status change writes a `SupportTicketEvent` for history.

## Endpoints
- `GET  /api/audit/logs` · `POST /api/audit/logs`
- `POST /api/audit/exports` (CSV body) · `GET /api/audit/exports/:exportId`
- `GET  /api/support/tickets` · `POST /api/support/tickets`
- `GET  /api/support/tickets/:ticketId`
- `POST /api/support/tickets/:ticketId/comments`
- `PATCH /api/support/tickets/:ticketId/status`

## Not done yet (see manifest knownLimitations)
No email notifications, no role-based visibility enforcement on internal notes
(visibility is stored + UI-separated but not access-controlled), no async export
for very large ranges. Native input execution remains disabled.
```

Part I — Handoff meta (doNotMerge)

`generated-remotedesk-pack14-merge-summary.md` — doNotMerge

```
# RemoteDesk Pack 14 — Merge Summary

**Pack:** API Audit Logging, Compliance Exports & Support Ticket Persistence
**Files:** 41 (32 SAFE_DIRECT_COPY, 6 REVIEW_REQUIRED, 3 doNotMerge meta)

## Build order
```

```
1. `packages/shared` — audit/* + support/* then patch shared `index.ts` + `errors/errorCodes.ts`; build.
2. `apps/api` — patch `schema.prisma` (5 models + back-relations), run `0014_audit_support` migration + `prisma generate`, drop in lib/repos/routes, patch `server.ts`.
3. `apps/web` — drop in `dashboard/audit/*` and `dashboard/support/*`, patch dashboard links.
4. `apps/desktop` — drop in the two clients, patch `session/criticalEvents.ts`.
5. Tests — `pnpm -r test`.

## REVIEW_REQUIRED (hand-merge)
- `packages/shared/src/index.ts` — add audit + support barrels.
- `packages/shared/src/errors/errorCodes.ts` — add 3 codes.
- `apps/api/prisma/schema.prisma` — add 5 models + back-relations on User/Device.
- `apps/api/src/server.ts` — mount /api/audit and /api/support.
- `apps/web/app/dashboard/page.tsx` — add 2 links.
- `apps/desktop/src/renderer/src/session/criticalEvents.ts` — wire audit calls.

## Known limitations
- No live email notifications on ticket activity.
- Internal-vs-public is stored + UI-separated but NOT access-controlled (no RBAC wired).
- Compliance export is synchronous; add async/streaming for very large ranges.
- `auditApi.export` assumes an apiClient `raw()` passthrough; add one or inline fetch.
- Native input execution remains disabled by design.

## Safety
- Audit metadata redacted before persistence (server) and before send (desktop), with an assert backstop.
- CSV export re-redacts, so exports never contain secrets (incl. device passwords).
- Desktop audit client fails silently — never blocks a session.

## Verify
- `pnpm -r typecheck && pnpm -r test`
- Manual: POST an audit log with an `Authorization` metadata key -> stored as [REDACTED].
- Manual: create a ticket, add a public + internal comment, transition open->resolved->closed.
- Manual: export a date range, confirm CSV downloads and contains no secrets.
```

`generated-remotedesk-pack14-code-review.md` — doNotMerge

```
# RemoteDesk Pack 14 — Code Review Notes

## Strengths
- Redaction is pure, shared, and triple-applied (desktop send, server write, CSV export) with an assert backstop.
- Status transitions are an explicit allow-list; closed is terminal and enforced server-side.
- Public vs internal comments are first-class and visually separated.
- Every ticket mutation writes a SupportTicketEvent for history.
- Tests import only real shared/copied files (no REVIEW_REQUIRED-only imports).

## Reviewer checklist
- [ ] `apiClient` needs a `raw()` (or inline fetch) for the CSV export download.
- [ ] Decide whether internal notes need RBAC before exposing to non-staff users.
- [ ] Confirm `User`/`Device` model + id fields match the migration FKs.
- [ ] Large-range exports: consider streaming/pagination + async job.
- [ ] Audit `list` caps at 200; add pagination if the UI needs more.

## Security
- No secrets persisted in audit metadata or exports; device passwords explicitly covered.
- No live email; no fake RBAC.
- Native input execution untouched and still disabled.
```

`generated-remotedesk-pack14-manifest.json` — doNotMerge

```
{
  "pack": "pack14",
  "title": "API Audit Logging, Compliance Exports & Support Ticket Persistence",
  "actualFileCount": 41,
  "safeDirectCopy": [
    "packages/shared/src/audit/types.ts",
    "packages/shared/src/audit/redaction.ts",
    "packages/shared/src/audit/exportCsv.ts",
    "packages/shared/src/audit/schema.ts",
    "packages/shared/src/audit/index.ts",
    "packages/shared/src/support/types.ts",
    "packages/shared/src/support/status.ts",
    "packages/shared/src/support/priority.ts",
    "packages/shared/src/support/schema.ts",
    "packages/shared/src/support/index.ts",
    "apps/api/prisma/migrations/0014_audit_support/migration.sql",
    "apps/api/src/lib/auditLogger.ts",
    "apps/api/src/lib/complianceExports.ts",
    "apps/api/src/lib/supportTickets.ts",
    "apps/api/src/repositories/auditRepository.ts",
    "apps/api/src/repositories/supportRepository.ts",
    "apps/api/src/routes/audit.routes.ts",
    "apps/api/src/routes/support.routes.ts",
    "apps/web/app/dashboard/audit/auditApi.ts",
    "apps/web/app/dashboard/audit/page.tsx",
    "apps/web/app/dashboard/support/supportApi.ts",
    "apps/web/app/dashboard/support/page.tsx",
    "apps/web/app/dashboard/support/[ticketId]/page.tsx",
    "apps/web/app/dashboard/support/support.module.css",
    "apps/desktop/src/renderer/src/services/auditClient.ts",
    "apps/desktop/src/renderer/src/services/supportClient.ts",
    "tests/audit/redaction.test.ts",
    "tests/audit/exportCsv.test.ts",
    "tests/audit/audit-contract.test.ts",
    "tests/support/status.test.ts",
    "tests/support/priority.test.ts",
    "tests/support/support-ticket-contract.test.ts",
    "docs/audit-compliance-support.md"
  ],
  "reviewRequired": [
    "packages/shared/src/index.ts",
    "packages/shared/src/errors/errorCodes.ts",
    "apps/api/prisma/schema.prisma",
    "apps/api/src/server.ts",
    "apps/web/app/dashboard/page.tsx",
    "apps/desktop/src/renderer/src/session/criticalEvents.ts"
  ],
  "doNotMerge": [
    "generated-remotedesk-pack14-merge-summary.md",
    "generated-remotedesk-pack14-code-review.md",
    "generated-remotedesk-pack14-manifest.json"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["zod", "@prisma/client", "express", "next", "react", "vitest"],
    "assumedFromPriorBatches": [
```

```
        "@remotedesk/shared AppError + ErrorCodes",
        "apps/api db/prisma client",
        "apps/api middleware: requireAuth, orgContext",
        "apps/web src/api/apiClient (api.get/post/patch + raw passthrough)",
        "existing Prisma User and Device models",
        "apps/desktop critical-event hooks (onLogin/onSessionStart/etc.)"
      ]
    },
    "knownLimitations": [
      "No live email notifications on ticket activity.",
      "Internal vs public comments are stored + UI-separated but NOT access-controlled (no RBAC wired).",
      "Compliance export is synchronous; add async/streaming for very large date ranges.",
      "auditApi.export assumes an apiClient raw() passthrough; add one or inline fetch.",
      "Native input execution remains disabled."
    ],
    "estimatedCompletionAfterCarefulMerge": "66-72%"
  }
```

Pack 14 done: 41 files. Append-only audit logging with mandatory redaction, secret-free compliance CSV exports over a date range, and fully persisted support tickets with public/internal comments, validated status transitions, and a per-ticket event history.

The clean bit: redaction is one pure shared function applied **three times** (desktop before send, server before write, CSV before export) with an `assertRedacted` backstop, so device passwords / tokens / cookies / auth headers / API keys can't reach storage or an export. Status transitions are an explicit allow-list (closed is terminal), and the desktop audit client fails silently so logging never breaks a session. No email, no fake RBAC, native input still disabled.

That's Packs 13 + 14 both filed. Next number = Pack 15. Bolo.